

Numpy



"NumPy es al cómputo científico en Python lo que el álgebra lineal es a las matemáticas: el lenguaje base sobre el que todo lo demás se construye."

Introducción

[NumPy](#) es la biblioteca fundamental para el cómputo numérico en Python. Su objeto central — el **array multidimensional** (`ndarray`) — permite operar sobre vectores y matrices de forma eficiente, con una sintaxis mucho más limpia que las listas nativas de Python.

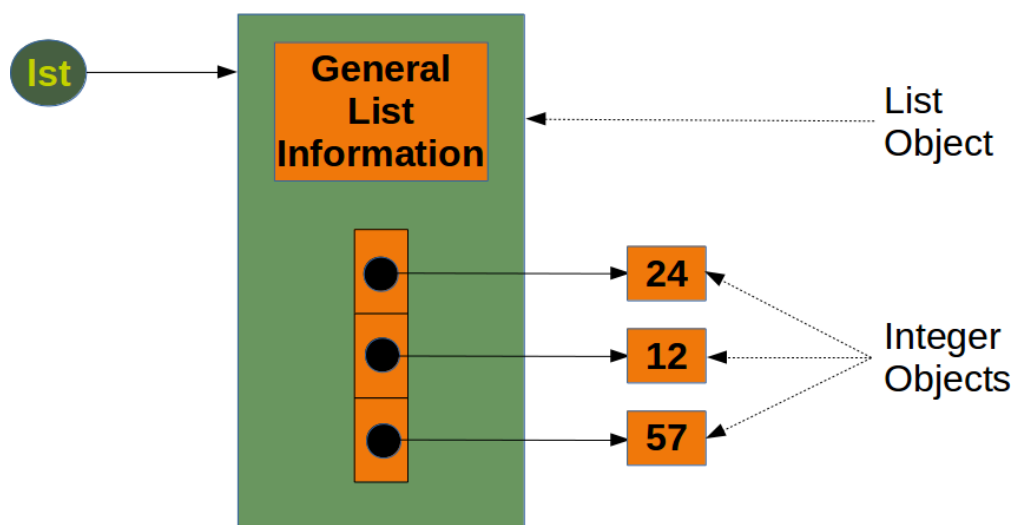
En este módulo aprenderás a:

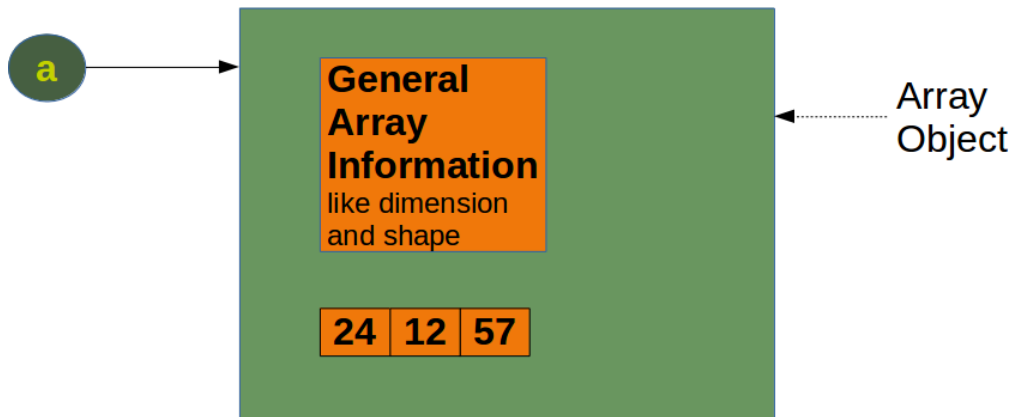
- Crear y manipular arrays de una y múltiples dimensiones
- Realizar operaciones aritméticas, estadísticas y de álgebra lineal
- Visualizar datos numéricos con **matplotlib** de forma básica

Python Lists vs Numpy Arrays

Antes de crear arrays, vale la pena entender por qué no basta con las listas de Python.

```
64 + 8 * len (lst) + + len (lst) * 28
```



$96 + n * 8 \text{ bytes}$


```
import numpy as np
import time
import sys

def arreglo_python(n):
    t1 = time.time()
    X = range(n)
    Y = range(n)
    Z = [X[i] + Y[i] for i in range(len(X)) ]
    return (time.time() - t1, sys.getsizeof(Z) )

def arreglo_numpy(n):
    t1 = time.time()
    X = np.arange(n)
    Y = np.arange(n)
    Z = X + Y
    return (time.time() - t1, sys.getsizeof(Z) )
```

```
# generar varios casos
for size_of_vec in [10,100,1000,10000,100000,1000000]:
    print(f"size of vector: {size_of_vec}")

    #list vs numpy
    t1, size1 = arreglo_python(size_of_vec)
    t2, size2 = arreglo_numpy(size_of_vec)

    # resultados
    print(f"python list -- time: {round(t1,8)} seg, size: {size1} bytes")
    print(f"numpy array -- time: {round(t2,8)} seg, size: {size2} bytes\n")
```

```

size of vector: 10
python list -- time: 8.82e-06 seg, size: 184 bytes
numpy array -- time: 8.13e-05 seg, size: 192 bytes

size of vector: 100
python list -- time: 1.86e-05 seg, size: 920 bytes
numpy array -- time: 1.931e-05 seg, size: 912 bytes

size of vector: 1000
python list -- time: 0.0001955 seg, size: 8856 bytes
numpy array -- time: 0.00024605 seg, size: 8112 bytes

size of vector: 10000
python list -- time: 0.00200582 seg, size: 85176 bytes
numpy array -- time: 0.00025821 seg, size: 80112 bytes

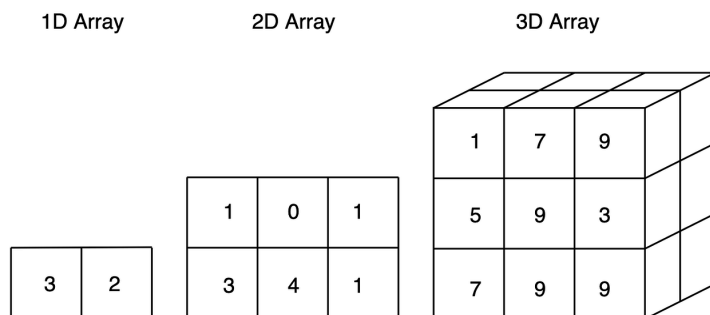
size of vector: 100000
python list -- time: 0.02143574 seg, size: 800984 bytes
numpy array -- time: 0.00159764 seg, size: 800112 bytes

size of vector: 1000000
python list -- time: 0.19724131 seg, size: 8448728 bytes
numpy array -- time: 0.01485848 seg, size: 8000112 bytes

```

✅ NumPy es ~2x más eficiente en memoria y hasta **50x más rápido**, para operaciones numéricas a gran escala.

Objetos en Numpy



El objeto principal es el `ndarray`. Puede ser 1D (vector), 2D (matriz) o nD (tensor).

```

import numpy as np

# Vector (1D)
v = np.array([1, 2, 3])

# Matriz (2D)
M = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

```

```
# Tensor (3D)
T = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(f"vector: shape={v.shape}, ndim={v.ndim}, dtype={v.dtype}")
print(f"matriz: shape={M.shape}, ndim={M.ndim}, dtype={M.dtype}")
print(f"tensor: shape={T.shape}, ndim={T.ndim}, dtype={T.dtype}")
```

```
vector: shape=(3,), ndim=1, dtype=int64
matriz: shape=(2, 3), ndim=2, dtype=int64
tensor: shape=(2, 2, 2), ndim=3, dtype=int64
```

Atributo	Descripción	Ejemplo
<code>.shape</code>	Dimensiones del array	<code>(3,)</code> , <code>(2,3)</code>
<code>.ndim</code>	Número de dimensiones	<code>1</code> , <code>2</code> , <code>3</code>
<code>.size</code>	Total de elementos	<code>6</code>
<code>.dtype</code>	Tipo de datos	<code>int64</code> , <code>float64</code>

Arrays especiales

```
import numpy as np

print("zeros (3x3):")
print(np.zeros((3, 3)))

print("\nones (2x4):")
print(np.ones((2, 4)))

print("\nidentidad (3x3):")
print(np.eye(3))

print("\narange (0 a 10, paso 2):")
print(np.arange(0, 10, 2))

print("\nlinspace (0 a 1, 5 puntos):")
print(np.linspace(0, 1, 5))

np.random.seed(42)
print("\nuniform aleatorio (seed=42):")
print(np.random.uniform(0, 10, size=6))
```

```

zeros (3x3):
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

ones (2x4):
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

identidad (3x3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

arange (0 a 10, paso 2):
[0 2 4 6 8]

linspace (0 a 1, 5 puntos):
[0.    0.25 0.5   0.75 1.   ]

uniform aleatorio (seed=42):
[3.74540119  9.50714306  7.31993942  5.98658484  1.5601864   1.5599452 ]

```

Operaciones

Aritméticas (element-wise)

Operaciones aritméticas

```

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)    # [5 7 9]
print(a - b)    # [-3 -3 -3]
print(a * b)    # [4 10 18]
print(a / b)    # [0.25 0.4 0.5]
print(a ** 2)   # [1 4 9]

```

```

[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4 0.5 ]
[1 4 9]

```

- `a * b` es multiplicación **elemento a elemento**, no producto matricial.
- Para producto matricial usa `np.dot(A, B)` o el operador `@`.

Comparación

```

a = np.array([1, 2, 3])
b = np.array([2, 2, 4])

print(a == b)    # [False True False]

```

```
print(a > b)    # [False False False]
print(a <= b)   # [ True  True  True]

[False  True False]
[False False False]
[ True  True  True]
```

Los resultados son arrays booleanos — útiles para **filtrar** datos:

```
# Seleccionar elementos mayores a 2
print(a[a > 2])  # [3]

[3]
```

Estadísticas

```
a = np.array([2, 4, 4, 4, 5, 5, 7, 9])

print(f"suma:      {np.sum(a)}")
print(f"media:     {np.mean(a):.2f}")
print(f"std:       {np.std(a):.2f}")
print(f"mínimo:    {np.min(a)}")
print(f"máximo:    {np.max(a)}")
print(f"mediana:   {np.median(a)}")

suma:      40
media:     5.00
std:       2.00
mínimo:    2
máximo:    9
mediana:   4.5
```

Indexación y Slicing

```
# --- 1D ---
a = np.array([10, 20, 30, 40, 50])

print("\n--- vector 1D ---")
print(a)

print("\nprimer elemento (a[0]):")
print(a[0])

print("\núltimo elemento (a[-1]):")
print(a[-1])

print("\nslice a[1:4]:")
print(a[1:4])

print("\ncada dos elementos a[::2]:")
print(a[::2])

# --- 2D ---
M = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
```

```

print("\n--- matriz 2D ---")
print(M)

print("\nprimera fila M[0, :]:")
print(M[0, :])

print("\nsegunda columna M[:, 1]:")
print(M[:, 1])

print("\nelemento (fila 1, col 2) M[1, 2]:")
print(M[1, 2])

print("\nprimeras dos filas M[:2, :]:")
print(M[:2, :])

print("\ndiagonal M.diagonal():")
print(M.diagonal())

```

```

--- vector 1D ---
[10 20 30 40 50]

primer elemento (a[0]):
10

último elemento (a[-1]):
50

slice a[1:4]:
[20 30 40]

cada dos elementos a[::2]:
[10 30 50]

--- matriz 2D ---
[[1 2 3]
 [4 5 6]
 [7 8 9]]

primera fila M[0, :]:
[1 2 3]

segunda columna M[:, 1]:
[2 5 8]

elemento (fila 1, col 2) M[1, 2]:
6

primeras dos filas M[:2, :]:
[[1 2 3]
 [4 5 6]]

diagonal M.diagonal():
[1 5 9]

```

Algebra Lineal

NumPy cubre las operaciones fundamentales del curso vía `np.linalg`:

```

import numpy as np

A = np.array([[1, 2],
              [3, 4]])

# --- operaciones básicas ---
print("matriz A:")
print(A)

print("\ntranspuesta A.T:")
print(A.T)

print("\ndeterminante:")
print(round(np.linalg.det(A), 4))

print("\ninversa:")
print(np.linalg.inv(A))

print("\ntraza:")
print(np.trace(A))

# --- valores y vectores propios ---
print("\n--- eigendecomposición ---")
vals, vecs = np.linalg.eig(A)
print(f"eigenvalues: {np.round(vals, 4)}")
print(f"eigenvectors:\n{np.round(vecs, 4)}")

# --- normas ---
print("\n--- normas ---")
print(f"norma frobenius: {np.linalg.norm(A, 'fro'):.4f}")
print(f"norma espectral: {np.linalg.norm(A, 2):.4f}")
print(f"rango: {np.linalg.matrix_rank(A)}")

# --- descomposiciones ---
print("\n--- descomposición SVD ---")
U, S, Vt = np.linalg.svd(A)
print(f"U:\n{np.round(U, 4)}")
print(f"valores singulares: {np.round(S, 4)}")
print(f"Vt:\n{np.round(Vt, 4)}")

# --- sistema de ecuaciones Ax = b ---
print("\n--- sistema Ax = b ---")
b = np.array([5., 7.])
x = np.linalg.solve(A, b)
print(f"b: {b}")
print(f"solución x: {np.round(x, 4)}")
print(f"verificación Ax: {np.round(A @ x, 4)}")

```

```

matriz A:
[[1 2]
 [3 4]]

transpuesta A.T:
[[1 3]
 [2 4]]

determinante:
-2.0

inversa:
[[-2.   1. ]
 [ 1.5 -0.5]]

traza:
5

--- eigendescomposición ---
eigenvalues: [-0.3723  5.3723]
eigenvectors:
[[-0.8246 -0.416 ]
 [ 0.5658 -0.9094]]

--- normas ---
norma frobenius:  5.4772
norma espectral:  5.4650
rango:           2

--- descomposición SVD ---
U:
[[-0.4046 -0.9145]
 [-0.9145  0.4046]]
valores singulares: [5.465 0.366]
Vt:
[[-0.576 -0.8174]
 [ 0.8174 -0.576 ]]

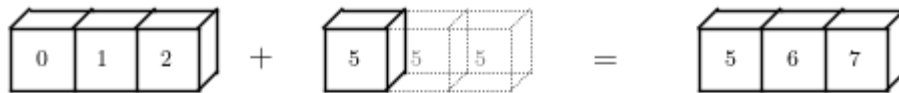
--- sistema Ax = b ---
b:      [5. 7.]
solución x: [-3. 4.]
verificación Ax: [5. 7.]

```

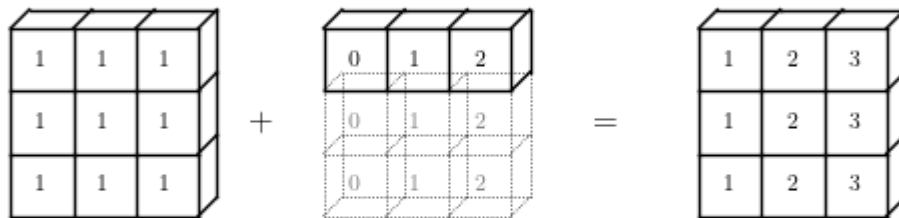
Broadcasting

Broadcasting permite operar entre arrays de **distinta forma** sin copiar datos. La regla básica: NumPy "expande" el array más pequeño para que las formas sean compatibles.

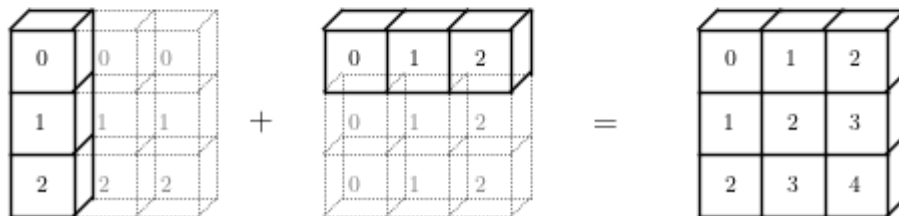
`np.arange(3) + 5`



`np.ones((3, 3)) + np.arange(3)`



`np.arange(3).reshape((3, 1)) + np.arange(3)`



```
import numpy as np

# --- escalar + vector ---
print("escalar + vector (np.arange(3) + 5):")
print(np.arange(3) + 5)

print("\nescalar * vector (np.arange(3) * 3):")
print(np.arange(3) * 3)

# --- matriz + vector (broadcasting por filas) ---
print("\n--- broadcasting por filas ---")
print("matriz ones(3,3) + arange(3):")
print(np.ones((3, 3)) + np.arange(3))

print("\nmatriz ones(3,3) * arange(3):")
print(np.ones((3, 3)) * np.arange(3))

# --- vector columna + vector fila → matriz ---
print("\n--- vector columna + vector fila ---")
col = np.arange(3).reshape((3, 1))
fil = np.arange(3)
print("columna:")
print(col)
print("\nfila:")
print(fil)
print("\nresultado col + fila:")
print(col + fil)

# --- comparaciones con broadcasting ---
print("\n--- comparaciones ---")
M = np.arange(9).reshape((3, 3))
print("matriz M:")
print(M)
print("\nM > 4:")
print(M > 4)
print("\nM[M > 4] (filtrado):")
```

```
print(M[M > 4])

# --- operaciones entre matrices compatibles ---
print("\n--- shapes compatibles ---")
A = np.ones((3, 3)) * 10
b = np.array([1, 2, 3])
print("A (3x3):")
print(A)
print("\nb (3,):")
print(b)
print("\nA / b:")
print(A / b)
```

```

escalar + vector (np.arange(3) + 5):
[5 6 7]

escalar * vector (np.arange(3) * 3):
[0 3 6]

--- broadcasting por filas ---
matriz ones(3,3) + arange(3):
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]

matriz ones(3,3) * arange(3):
[[0. 1. 2.]
 [0. 1. 2.]
 [0. 1. 2.]]

--- vector columna + vector fila ---
columna:
[[0]
 [1]
 [2]]

fila:
[0 1 2]

resultado col + fila:
[[0 1 2]
 [1 2 3]
 [2 3 4]]

--- comparaciones ---
matriz M:
[[0 1 2]
 [3 4 5]
 [6 7 8]]

M > 4:
[[False False False]
 [False False  True]
 [ True  True  True]]

M[M > 4] (filtrado):
[5 6 7 8]

--- shapes compatibles ---
A (3x3):
[[10. 10. 10.]
 [10. 10. 10.]
 [10. 10. 10.]]

b (3,):
[1 2 3]

A / b:
[[10.         5.         3.33333333]
 [10.         5.         3.33333333]
 [10.         5.         3.33333333]]

```

Referencias

1. [Quickstart tutorial-numpy](#)